



Instituto Infnet

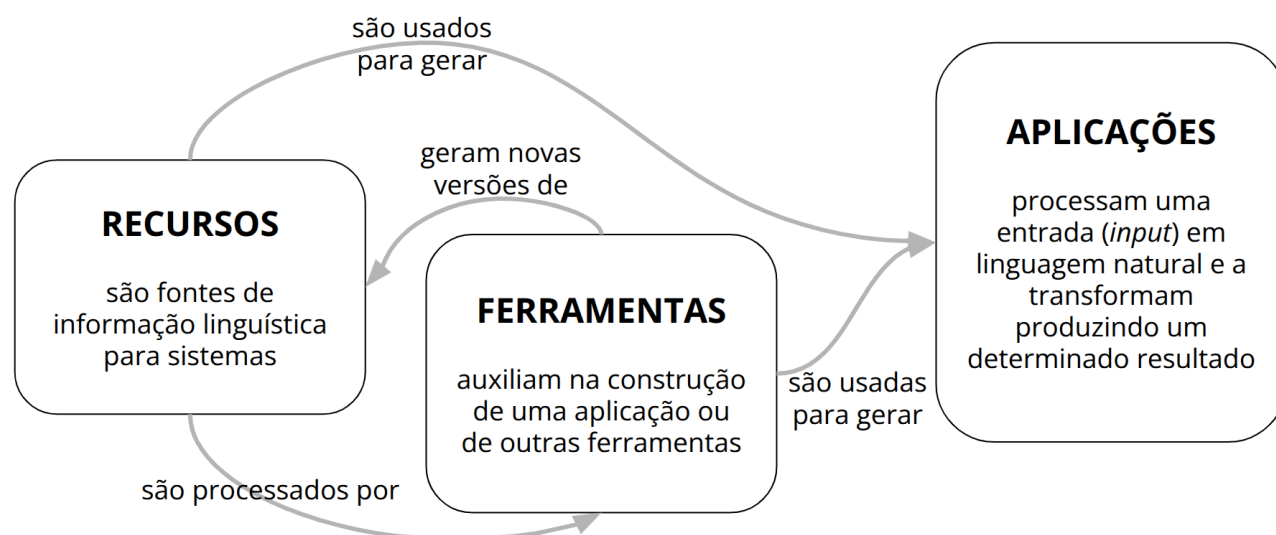
2º Ciclo - Fundamentos do Processamento de Dados

Python para Processamento de Dados [26E2_4]

Strings (str) - Continuação

Uma aplicação em Processamento de Linguagem Natural (PLN)

O Processamento de Linguagem Natural (PLN) é um campo de pesquisa que tem como objetivo investigar e propor métodos e sistemas de processamento computacional da linguagem humana. Na área da Ciência da Computação, PLN está ligado à área de Inteligência Artificial (IA) e também está intrinsecamente relacionada à Linguística Computacional [1].

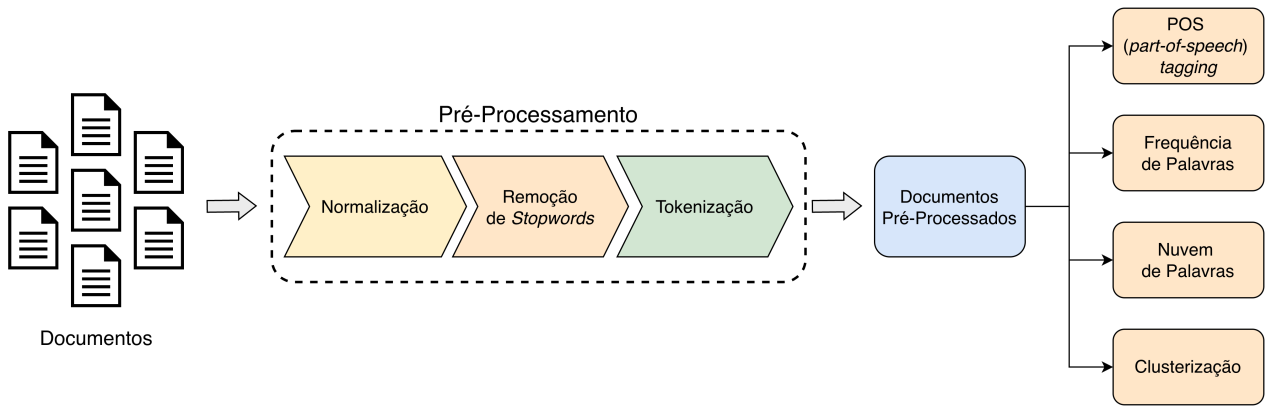


Para desenvolver praticamente qualquer aplicação de PLN, é necessário realizar fases/etapas que convencionamos chamar de pré-processamento. Nesse pré-processamento, algumas tarefas usuais são: segmentação do texto em sentenças (sentencição), separação de palavras (tokenização), tokenização em subpalavras (vetorização de subtokens), normalização de palavras (lematização e radicalização), entre outras [1].

Etapas de Pré processamento em PLN



Exemplo prático



```
texto_ata_copom = '''Atualização da conjuntura econômica do cenário do Copom ambiente externo se mantém adverso Apesar da  
print(texto_ata_copom)
```

Atualização da conjuntura econômica do cenário do Copom ambiente externo se mantém adverso Apesar da atenuação do estresse envolv

Pesquisa de strings

```

stopwords_br = [
    # Artigos e contrações
    'a', 'o', 'as', 'os', 'um', 'uma', 'uns', 'umas', 'ao', 'à', 'aos', 'às',
    'do', 'da', 'dos', 'das', 'no', 'na', 'nos', 'nas', 'pelo', 'pela', 'pelos', 'pelas',

    # Pronomes
    'eu', 'tu', 'ele', 'ela', 'nós', 'vós', 'eles', 'elas', 'me', 'mim', 'comigo',
    'te', 'ti', 'contigo', 'se', 'si', 'consigo', 'lhe', 'nos', 'vos', 'lhes',
    'meu', 'minha', 'meus', 'minhas', 'teu', 'tua', 'teus', 'tuas', 'seu', 'sua',
    'seus', 'suas', 'nosso', 'nossa', 'nossos', 'nossas', 'vosso', 'vossa', 'vossos', 'vossas',

    # Preposições
    'de', 'em', 'por', 'para', 'com', 'sem', 'sob', 'sobre', 'entre', 'até',
    'desde', 'contra', 'perante', 'através', 'além', 'dentro', 'fora', 'perto',
    'longe', 'durante',

    # Conjunções
    'e', 'mas', 'ou', 'porque', 'pois', 'que', 'se', 'como', 'quando', 'embora',
    'porém', 'todavia', 'contudo', 'então', 'também', 'apesar', 'caso', 'além disso',
    'portanto', 'logo', 'ainda que', 'a fim de', 'com o objetivo de',

    # Verbos auxiliares e comuns
    'ser', 'estar', 'ter', 'haver', 'ir', 'vir', 'fazer', 'poder', 'dever',
    'querer', 'saber', 'está', 'estão', 'tem', 'tenho', 'é',

    # Demonstrativos
    'esse', 'essa', 'isso', 'este', 'esta', 'isto', 'aquele', 'aquela', 'aquilo',
    'aqueles', 'aquelas', 'deste', 'desta', 'destes', 'destas', 'disso', 'daquilo',
    'nisto', 'naquilo',

    # Localizadores
    'lá', 'aqui', 'ali', 'onde', 'aonde',

    # Numericas
    'zero', '1', '2', '3', '4', '5', '6', '7', '8', '9', '0'
]

# Encapsulando o pre-processamento
def pre_processamento(stopwords, texto):
    """
    Definicao da funcao
    Etapas: Normalizacao, Tokenizacao (anagramas) e Remocao de Stopwords

    return
    Tupla:
        1o: lista com os tokens pre-processados
        2o: texto (str) pre-processada
    """
    texto = texto.lower() #Normalizacao
    tokens = texto.split() #Tokenizacao (anagramas)
    # list comprehension
    tokens = [token for token in tokens if token not in stopwords] # remocao das stopwords

    return tokens, ' '.join(tokens)

```

```

lista_tokens_preprocessado, texto_preprocessado = pre_processamento(stopwords=stopwords_br, texto=texto_ata_copom)
print(lista_tokens_preprocessado)
print(texto_preprocessado)

```

```

['atualização', 'conjuntura', 'econômica', 'cenário', 'copom', 'ambiente', 'externo', 'mantém', 'adverso', 'atenuação', 'estresse
atualização conjuntura econômica cenário copom ambiente externo mantém adverso atenuação estresse envolvendo bancos eua europa li

```

```

help(str.find) #str.rfind

```

Help on method_descriptor:

```

find(...) unbound builtins.str method
S.find(sub[, start[, end]]) -> int

```

Return the lowest index in S where substring sub is found, such that sub is contained within S[start:end]. Optional arguments start and end are interpreted as in slice notation.

Return -1 on failure.

```
# outra funcao de busca de str: index e rindex
help(str.index)
```

Help on method_descriptor:

```
index(...) unbound builtins.str method
    S.index(sub[, start[, end]]) -> int
```

Return the lowest index in S where substring sub is found, such that sub is contained within S[start:end]. Optional arguments start and end are interpreted as in slice notation.

Raises ValueError when the substring is not found.

```
texto_preprocessado.find('copom')
```

41

```
texto_preprocessado.index('copom')
```

41

```
texto_preprocessado.find('zero')
print('string nao encontrada')
```

string nao encontrada

```
texto_preprocessado.index('zero')
print('string nao encontrada')
```

 Execution error

ValueError: substring not found

```
-----
ValueError                                Traceback (most recent call last)
Cell In[25], line 1
----> 1 texto_preprocessado.index('zero')
      2 print('string nao encontrada')
```

ValueError: substring not found

[Hide error details](#)

```
texto_preprocessado.rfind('copom')
```

14902

```
texto_preprocessado.rindex('copom')
```

14902

Posicionamento de strings

```
# funcoes de ajudam a apresentar strings
```

```
animal = 'tigre'  
  
'X' + animal + 'X'
```

```
'XtigreX'
```

```
# center  
help(str.center)
```

Help on method_descriptor:

```
center(self, width, fillchar=' ', /) unbound builtins.str method  
    Return a centered string of length width.  
  
    Padding is done using the specified fill character (default is a space).
```

```
'X' + animal.center(10) + 'X'
```

```
'X  tigre  X'
```

```
'X' + animal.center(10, '.') + 'X'
```

```
'X..tigre...X'
```

```
'X' + animal.center(10, '_') + 'X'
```

```
'X__tigre__X'
```

```
# ljust  
help(str.ljust)
```

Help on method_descriptor:

```
ljust(self, width, fillchar=' ', /) unbound builtins.str method  
    Return a left-justified string of length width.  
  
    Padding is done using the specified fill character (default is a space).
```

```
animal.ljust(10)
```

```
'tigre      '
```

```
animal.ljust(10, '.')
```

```
'tigre.....'
```

```
# rjust
help(str.rjust)
```

Help on method_descriptor:

```
rjust(self, width, fillchar=' ', /) unbound builtins.str method
    Return a right-justified string of length width.

    Padding is done using the specified fill character (default is a space).
```

```
animal.rjust(10)
```

```
'      tigre'
```

```
animal.rjust(10, '.')
```

```
'.....tigre'
```

```
# outra forma de posicionamento com python eh atraves f-string
```

```
print(f'X{animal:^10}X') # equivalente ao center
print(f'{animal:<10}') # equivalente ao ljust
print(f'{animal:>10}') # equivalente ao rjust
```

```
X tigre  X
tigre
      tigre
```

```
print(f'{animal:.>10}') # equivalente ao rjust passando o '.' como preenchimento
```

```
.....tigre
```

```
# separacao de strings -> split
print(animal.split('g'))
# substituicao de string -> replace
print(animal.replace('g', 'G'))
# remocao de espacos em branco -> strip
exemplo_remocao_espaco_em_branco = 'python'.center(20)
print(exemplo_remocao_espaco_em_branco)
print(exemplo_remocao_espaco_em_branco.strip())
```

```
['ti', 're']
tiGre
      python
python
```

Validação por tipo de conteúdo

```
# isalnum
help(str.isalnum)
```

Help on method_descriptor:

```
isalnum(self, /) unbound builtins.str method
    Return True if the string is an alpha-numeric string, False otherwise.
```

A string is alpha-numeric if all characters in the string are alpha-numeric and there is at least one character in the string.

```
exemplo_isalnum_1 = '123'
exemplo_isalnum_2 = 'alon mundo python v2'
exemplo_isalnum_3 = ''
print(exemplo_isalnum_1, exemplo_isalnum_1.isalnum())
print(exemplo_isalnum_2, exemplo_isalnum_2.isalnum())
print(exemplo_isalnum_3, exemplo_isalnum_3.isalnum())
```

```
123 True
alon mundo python v2 False
False
```

```
# isalpha
help(str.isalpha)
```

Help on method_descriptor:

isalpha(self, /) unbound builtins.str method
Return True if the string is an alphabetic string, False otherwise.

A string is alphabetic if all characters in the string are alphabetic and there is at least one character in the string.

```
exemplo_isalpha_1 = '123'
exemplo_isalpha_2 = 'alo mundo'
exemplo_isalpha_3 = 'python'
exemplo_isalpha_4 = ''

print(exemplo_isalpha_1, exemplo_isalpha_1.isalpha())
print(exemplo_isalpha_2, exemplo_isalpha_2.isalpha())
print(exemplo_isalpha_3, exemplo_isalpha_3.isalpha())
print(exemplo_isalpha_4, exemplo_isalpha_4.isalpha())
```

```
123 False
alo mundo False
python True
False
```

```
# isdigit
help(str.isdigit)
```

Help on method_descriptor:

isdigit(self, /) unbound builtins.str method
Return True if the string is a digit string, False otherwise.

A string is a digit string if all characters in the string are digits and there is at least one character in the string.

```
exemplo_isdigit_1 = '771'
exemplo_isdigit_2 = '771.63'
exemplo_isdigit_3 = '+771'
exemplo_isdigit_4 = '-771'
exemplo_isdigit_5 = ''

print(exemplo_isdigit_1, exemplo_isdigit_1.isdigit())
print(exemplo_isdigit_2, exemplo_isdigit_2.isdigit())
print(exemplo_isdigit_3, exemplo_isdigit_3.isdigit())
print(exemplo_isdigit_4, exemplo_isdigit_4.isdigit())
print(exemplo_isdigit_5, exemplo_isdigit_5.isdigit())
```

```
771 True
771.63 False
+771 False
-771 False
False
```

```
# isnumeric
help(str.isnumeric)
```

Help on method_descriptor:

isnumeric(self, /) unbound builtins.str method

Return True if the string is a numeric string, False otherwise.

A string is numeric if all characters in the string are numeric and there is at least one character in the string.

```
exemplo_isnumeric_1 = '771'
exemplo_isnumeric_2 = '771.63'
exemplo_isnumeric_3 = '+771'
exemplo_isnumeric_4 = '-771'
exemplo_isnumeric_5 = ''
```

```
print(exemplo_isnumeric_1, exemplo_isnumeric_1.isnumeric())
print(exemplo_isnumeric_2, exemplo_isnumeric_2.isnumeric())
print(exemplo_isnumeric_3, exemplo_isnumeric_3.isnumeric())
print(exemplo_isnumeric_4, exemplo_isnumeric_4.isnumeric())
print(exemplo_isnumeric_5, exemplo_isnumeric_5.isnumeric())
```

```
771 True
771.63 False
+771 False
-771 False
False
```

```
# novem em tibetano
# Usando o escape unicode
nove_tibetano = "\u0f29"
print(nove_tibetano) # Saída: ࠋ

# Verificando se é numérico
print(nove_tibetano.isdigit())
print(nove_tibetano.isnumeric())
```

```
ࠋ
True
True
```

```
# um terço
# Usando o escape unicode
terco = "\u2153"
print(terco)

# Verificando se é numérico
print(terco.isdigit())
print(terco.isnumeric())
```

```
⅓
False
True
```

Outras funções uteis


```
# startswith
help(str.startswith)
```

Help on method_descriptor:

```
startswith(...) unbound builtins.str method
    S.startswith(prefix[, start[, end]]) -> bool

    Return True if S starts with the specified prefix, False otherwise.
    With optional start, test S beginning at that position.
    With optional end, stop comparing S at that position.
    prefix can also be a tuple of strings to try.
```

```
#
help(str.endswith)
```

Help on method_descriptor:

```
endswith(...) unbound builtins.str method
    S.endswith(suffix[, start[, end]]) -> bool

    Return True if S ends with the specified suffix, False otherwise.
    With optional start, test S beginning at that position.
    With optional end, stop comparing S at that position.
    suffix can also be a tuple of strings to try.
```

```
# Melhorando a funcao de pre-processamento
def pre_processamento_v2(stopwords, texto):
```

```
    ...
    Definicao da funcao
    Etapas: Normalizacao, Tokenizacao (anagramas) e Remocao de Stopwords

    return
    Tupla:
        1o: lista com os tokens pre-processados
        2o: texto (str) pre-processada
    ...
```

```
texto = texto.lower() #Normalizacao
tokens = texto.split() #Tokenizacao (anagramas)
```

```
tokens = [token.replace(',', ' ').replace('.', ' ').replace('; ', ' ') for token in tokens if (token.endswith(',')) or (token.endswith('; '))] # retirando tags html
tokens = [token for token in tokens if (not token.startswith('<')) and (not token.endswith('>'))] # retirando tags html
tokens = [token for token in tokens if token.find('class=') == -1] # retirando o 'class='
tokens = [token for token in tokens if not token.isnumeric()] # tirando numeros
tokens = [token for token in tokens if token.isalpha()] # considerando apenas palavras
```

```
tokens = [token for token in tokens if token not in stopwords] # remocao das stopwords
tokens = [token for token in tokens if len(token) > 2] # considerando tokens com pelo menos tres caracteres
```

```
return tokens, ' '.join(tokens)
```

```
texto_ata_255 = '<div id="atacompleta"><div id="ataconteudo"><h3 class="secao">A) Atualização da conjuntura econômica e d
print(texto_ata_255)
```

```
<div id="atacompleta"><div id="ataconteudo"><h3 class="secao">A) Atualização da conjuntura econômica e do cenário do Copom<sup>1<
```

```
lista_tokens_preprocessado, texto_preprocessado = pre_processamento_v2(stopwords=stopwords_br, texto=texto_ata_255)
lista_tokens_preprocessado[:10]
```

```
['adverso',
 'momento',
 'monitoramento',
 'entretanto',
 'incertos',
 'paralelo',
 'metas',
 'resiliente',
 'países',
 'inflacionário']
```

```
lista_tokens_preprocessado, texto_preprocessado = pre_processamento(stopwords=stopwords_br, texto=texto_ata_255)
lista_tokens_preprocessado[:10]
```

```
['<div',
 'id="atacompleta"><div',
 'id="ataconteudo"><h3',
 'class="secao">a)',
 'atualização',
 'conjuntura',
 'econômica',
 'cenário',
 'copom<sup>1</sup></h3><p',
 'class="paragrafo">1.']
```

Conteúdo para aprofundamento

[Documentação oficial sobre strings](#)